T16: A Decode-Cost-Driven Trellis Format for Running a 552B Mixture of Experts on Two Consumer GPUs

Aleksey Smolenchuk
lxe@lxe.co
dsx project, https://github.com/lxe/dsx

October 2026

Abstract

DeepSeek-V4.1-Flash-1017 is a 552B-parameter mixture of experts, and 98.5% of its weights are 15,360 routed experts. To run it on a desktop with two 24 GB RTX 3090s and 128 GB of RAM, those experts must take about 1.9 bits per weight. They must also decode fast in two very different places: as single-token GEMVs on the GPUs, and as AVX-VNNI loops on the CPU for the roughly 12,600 experts that live in RAM. We present T16, an expert format built from a measured decode-cost model of both processors. T16 keeps the bitshift trellis of QTIP and ExLlamaV3 but replaces its codebook. A weight's value comes from one 16-bit multiply and a two-byte sum, which makes dequantization and the dot product one fused integer operation on both processors, at 4% more mean squared error than ExLlamaV3's codebook on Gaussian data. Positions inside each tile are column-major, which costs no measurable quality and lets one file feed fast row-walking kernels. On the quantizer side we contribute a dense fractional-bit Viterbi kernel that is bit-identical to ExLlamaV3's and 5.1× faster at 1.5 bits. We also identify a systematic output shrinkage of MSE-optimal trellis reconstructions (the plain gain is $1 -$ relative MSE) and correct it for free in the stored output scales; on a half-T16 model this took KL divergence from 0.31 to 0.25. With experts at 1.5–2.5 bits, the full model scores KL 0.194 against the FP8 model on the Lucebox evaluation, 28% lower than the same-size build with Unsloth's UD-IQ1_M experts (0.272). Its expert kernels are 20–34% faster on the CPU and 22% faster on the GPU than that build's, and it decodes at 34.9 tokens/s on the two RTX 3090s. Quantizing all 15,360 experts took 6.1 hours on one rented GPU for $16.

1 Introduction

Large mixture-of-experts (MoE) models put most of their weights in experts that each token uses only rarely. DeepSeek-V4.1-Flash-1017 [6] (hereafter *V4.1*) has 40 MoE layers with 384 routed experts each, plus one shared expert. A router picks 6 routed experts per token per layer. Each routed expert is a SwiGLU block with three matrices: gate and up (2304×5120) and down (5120×2304). Together the routed experts hold $40 \cdot 384 \cdot 3 \cdot 5120 \cdot 2304 = 543.6$B of the model's 552B weights. The model also carries a 196B Engram memory, a table of n-gram embeddings read by lookup rather than matrix multiply. We leave that table at 4–6 bits and do not discuss it further.

dsx [19] is an inference engine written for this one model and one class of machine: two 24 GB GPUs, 128 GB of RAM and an NVMe drive. Figure 1 shows how it uses that machine. The dense half of the network and the hottest experts run on the GPUs, about 12,600 experts live in a RAM arena where the CPU computes them in place, and the coldest are read from the file. Each token touches 240 expert calls (6 per layer), about 2 GB of expert weights at the bit widths used here. About 74% of expert picks are served from VRAM, and the CPU's share sets decode speed.

This placement fixes what the expert format must do. It must reach about 1.9 bits per weight to fit, and at that rate it must keep quality close to the FP8 original. Most importantly, it must decode at memory speed on *both* processors, because the same file is read by GPU kernels for resident experts and by CPU kernels for the arena. Existing formats fall short on one side or the other. llama.cpp's IQ codebooks [9], as mixed by Unsloth's UD builds [26], decode reasonably on both but trail trellis codes in accuracy. ExLlamaV3's trellis codes [23, 25] come close to the Gaussian rate-distortion bound, but ExLlamaV3 keeps every weight in VRAM, and its codebook costs a 32-bit multiply per weight, which desktop CPUs execute slowly.

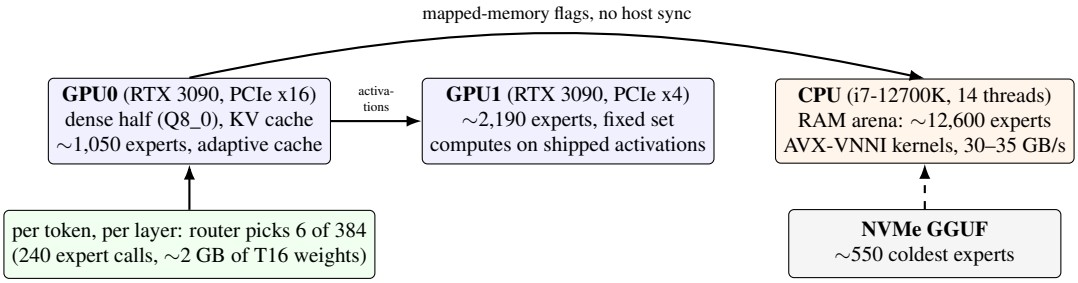


Figure 1: Where V4.1's 15,360 routed experts are computed during decode in dsx on 2×RTX 3090 + 128 GB RAM. The expert format is read by GPU kernels (resident experts) and CPU kernels (the arena), so it has to decode fast on both.

Contributions.

- A decode-cost model for single-token expert GEMVs on an RTX 3090 and an i7-12700K, built from measured per-instruction throughputs, and the per-weight instruction budget it implies at 1.5–2.5 bits (Section 3).
- **T16**, a trellis codebook chosen under that budget: a 16-bit multiply and a two-byte sum, which fuse with the activation dot product into `vpdpbusd` on the CPU and `dp4a` on the GPU. We pair it with column-major tile positions, which cost no quality. T16 kernels beat every IQ format of UD's mix on both processors (Section 4).
- A dense fractional-bit Viterbi kernel for ExLlamaV3's quantizer: bit-identical output, 5.1× faster at 1.5 bits, and full-model quantization 2.1× cheaper (Section 5).
- An analysis of the output shrinkage of MSE-optimal trellis reconstructions, and a free correction in the stored output scales (Section 5.1).
- A routing-aware calibration set for a 384-expert MoE: the model's own reasoning, scrubbed coding-agent transcripts and public agent trajectories, topped up with probe sequences until rare experts get enough tokens (Section 6).
- End-to-end results for a 552B model on two RTX 3090s, with quality, speed and quantization cost (Section 7).

2 Background and related work

Weight-only post-training quantization. GPTQ [8] rounds weights one input column at a time, feeding each column's error forward through the inverse of the input Hessian $H = \mathbb{E}[xx^\top]$. QuIP [3] recast this as LDLQ, rounding with the LDL factor of H, and introduced *incoherence processing*: random orthogonal transforms that make weights near Gaussian and spread outliers. QuIP# [22] used fast randomized Hadamard transforms and an 8-dimensional E_8 lattice codebook (E8P). QuaRot [2] rotates activations in the same spirit. Vector and additive codebooks (AQLM, VPTQ, PV-Tuning) [7, 12, 14] reach 2 bits with large learned codebooks, at substantial encoding cost. HIGGS [15] uses data-free Gaussian-optimal grids after Hadamard rotation. GSQ [5] learns scalar assignments with Gumbel-softmax sampling.

Trellis-coded quantization. Trellis-coded quantization (TCQ) [16] quantizes a whole sequence jointly with the Viterbi algorithm [27]. QTIP [23] made TCQ practical for LLMs with a *bitshift* trellis. Each weight's codeword index is a sliding L-bit window over a bit stream that advances K bits per weight, so decoding is parallel. The codebook is computed from the window by a few integer instructions (1MAD, 3INST) or a small table (HYB), and QTIP reports Gaussian MSE within about 10% of the rate-distortion bound at 2 bits. ExLlamaV3 (EXL3) [25] is a streamlined QTIP: 16×16 tiles with $L = 16$, 128-wide Hadamard transforms on both sides with per-channel scales, LDLQ over 16-row bands, and half-integer rates. Its default codebook (`mul1`) multiplies the window by a 32-bit constant and sums the product's four bytes. Q-Palette [11] and YAQA [24] extend QTIP with fractional rates, mixed schemes and Hessians of the full-model KL. XOR-Trellis [18] drops the Hadamard in favor of curvature-aware palettes. ik_llama.cpp's IQ*_KT

RTX 3090 (82 SMs, ∼1.74 GHz at 285 W), lanes / clock / SM			i7-12700K P-core, instructions / clock	
SHF, LOP3, PRMT	ALU	64	`vpshufb`, `vpsrld`	2
IMAD, IDP4A, HFMA2	FMA-heavy	64	`vpmullw` (16-bit)	1.6
FFMA	FMA	128	`vpdpbusd` (AVX-VNNI)	1.6
LDS, data-dependent index	shared	16	`vpmulld` (32-bit)	0.74
int8 `mma` m16n8k32	tensor	0.2 / clk	`vpgatherdd` (8 dwords)	0.17

Table 1: Measured instruction throughput (`bench/gpu_isa.cu`, `bench/cpu_isa.c`). E-cores run the vector instructions about 4× slower and add little.

types [10] code independent 8-weight groups from 16-bit seeds without a Viterbi search. Our work keeps EXL3's quantizer and replaces the codebook. The choice is driven by measured decode cost on desktop hardware rather than by MSE alone.

Running MoE models on consumer hardware. llama.cpp [9] offloads experts to the CPU with its ggml backend, and its IQ formats (8-weight codebook entries from global tables) underlie Unsloth's per-tensor "dynamic" mixes [26]. KTransformers [4] splits MoE inference across CPU and GPU. dsx [19] places experts by measured routing frequency across GPUs and a RAM arena, with no host synchronization inside a token. Its first builds used scalar ternary, 2-bit and 3-bit tiers from the Xolotl pipeline [17] and reached KL 0.373 at 148 GB. Unsloth's UD-IQ1_M experts did markedly better at the same size (KL 0.272), and our analysis traced much of the difference to how well they transfer from calibration text to new text. That observation motivated T16.

3 A decode-cost model for expert GEMVs

In decode, each expert matrix multiplies one activation vector, so its runtime is the larger of the time to read its bytes and the time to execute its per-weight instructions on the busiest execution pipe. We measured both sides on the target machine. GPU throughput was measured with one 1024-thread block per SM and `clock64`. CPU throughput was measured per core clock with eight independent dependency chains, calibrated by a dependent add chain (Table 1).

Budget. At ∼850 GB/s the GPU streams 3.4 trillion 2-bit weights per second. Matching that with 82 SMs leaves about 2.2 instructions per weight on each of the ALU and FMA-heavy pipes (1.6 at 1.5 bits, 2.7 at 2.5 bits) before decode, not memory, sets the time. The CPU's 43 GB/s across eight P-cores leaves about 4.5 weights per core clock at 2 bits. Two consequences follow. First, a data-dependent table lookup costs the GPU four ALU instructions' worth of throughput, and a gather yields only 1.4 entries per clock on a P-core, so a lookup code pays only if one lookup yields at least four weights. Second, a 32-bit multiply per weight (EXL3's `mul1`) runs at 0.74 per clock on the CPU, which by itself limits a core to below the RAM rate.

The floor. A bitshift trellis needs at least one window extraction (a funnel shift), one codebook evaluation and its share of the dot product per weight. If the codebook value is affine in a sum of bytes of an integer product, the dot product becomes exact integer arithmetic,

$$\sum_w f(s_w)\,x_w \;=\; \alpha\Big(\sum_w \sum_{\text{bytes } j} p_{w,j}\,x_w\Big) + \beta \sum_w x_w, \qquad p_w = (s_w \cdot M) \bmod 2^{16}, \tag{1}$$

which `vpdpbusd` (CPU) and `dp4a` (GPU) evaluate directly on the product bytes and int8 activations. With two consecutive rows' products packed by one PRMT and dot-multiplied with $[x_k\,x_k\,x_{k+1}\,x_{k+1}]$, the GPU cost is 1.5 ALU plus 1.5 FMA-heavy instructions per weight, inside the budget.

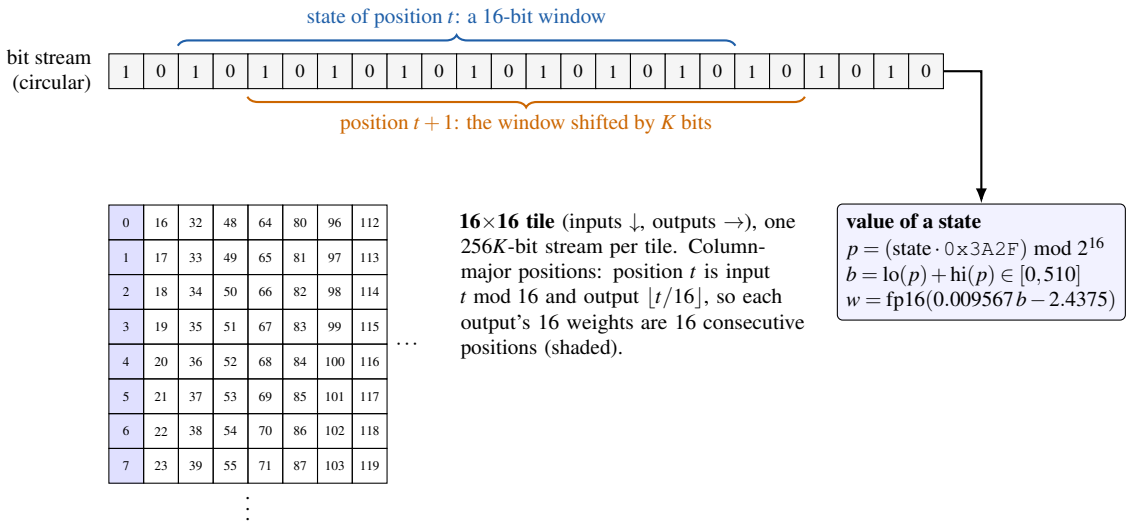


Figure 2: T16's storage. Each weight's state is a 16-bit window of its tile's circular bit stream. Consecutive positions overlap by $16-K$ bits, so a weight costs K bits; at half-integer K, even and odd positions alternate $\lfloor K \rfloor$ and $\lfloor K \rfloor + 1$ bits. The value is a two-byte sum of a 16-bit product (top right). Positions are column-major inside the tile (bottom).

4 The T16 format

T16 is EXL3's storage format with a different value function and position order (Figure 2). An expert matrix W (k inputs, n outputs) computes

$$y = s_v \odot \mathcal{H}_{128}\left(\mathcal{H}_{128}(x \odot s_u)\, W_{\text{inner}}\right), \tag{2}$$

where $\mathcal{H}_{128}$ is the orthonormal Walsh-Hadamard transform over blocks of 128, $s_u \in \mathbb{R}^k$ and $s_v \in \mathbb{R}^n$ are per-channel scales (fp16), and W_{inner} is stored as 16×16 tiles. Each tile is one circular bit stream of $256K$ bits, decoded at $L = 16$.

The codebook. A state s maps to $p = (s \cdot \texttt{0x3A2F}) \bmod 2^{16}$, $b = (p \,\&\, \texttt{0xFF}) + (p \gg 8)$, and $w = \text{fp16}(\text{fma}(1024 + b,\ \texttt{0x20e6},\ \texttt{0xca1e}))$. That is $0.009567\,b - 2.4375$ rounded to fp16, one of 511 values, approximately Gaussian because b sums two roughly uniform bytes. On the CPU, `vpmullw` multiplies 16 windows in one instruction (against 8 for `vpmulld`, at twice the throughput), and `vpdpbusd` takes the two bytes against the activations, two weights per 32-bit lane (Equation (1)). We searched the multiplier, byte-sum width and table-based alternatives with Viterbi on i.i.d. Gaussian tiles (Figure 3). T16's MSE is 0.0665 against `mul1`'s 0.0640 at 2 bits, a 4% increase worth about 0.03 bits per weight. The constant matters: `0xD12D` is 11% worse. Shuffle-only nibble tables are 21% worse.

Column-major positions. EXL3 orders a tile's 256 positions in the fragment order of the `mma.m16n8k16` tensor-core instruction, which suits batched GEMMs. For single-token GEMVs we want each output's 16 weights in 16 consecutive positions, so that a lane walks one output column with plain window shifts. Because LDLQ quantizes 16-row bands and the Viterbi search sees a 256-weight sequence of near-Gaussian values after the Hadamard transforms, the order should not matter for quality. Table 2 confirms it: with only ExLlamaV3's permutation swapped, calibration error agrees within 0.3%, and the evaluation ratios (+2.7%, −1.3%) are per-expert noise, with medians of 1.00.

Kernels. The GPU kernel gives each lane one output column of a tile pair and walks down the inputs. A running pointer and one 16-byte activation load serve 8 rows; the windows of a column come from one 64-bit view of the stream; and two rows' products are packed per `dp4a` with int8 activations (one scale per 256 inputs). SASS shows 1.66 ALU and 1.72 FMA-heavy instructions per weight, against the model's 1.5 + 1.5. The kernel is bound by load latency rather than by decode math. On the CPU, the column views come from byte-aligned 16-byte loads and one `vpshufb` per four columns (tile words byte-swapped once at load), then one shift, one `vpmullw` and one `vpdpbusd` per weight class. Figure 4 compares both with dsx's kernels for the IQ formats of UD's mix, measured interleaved in one session.

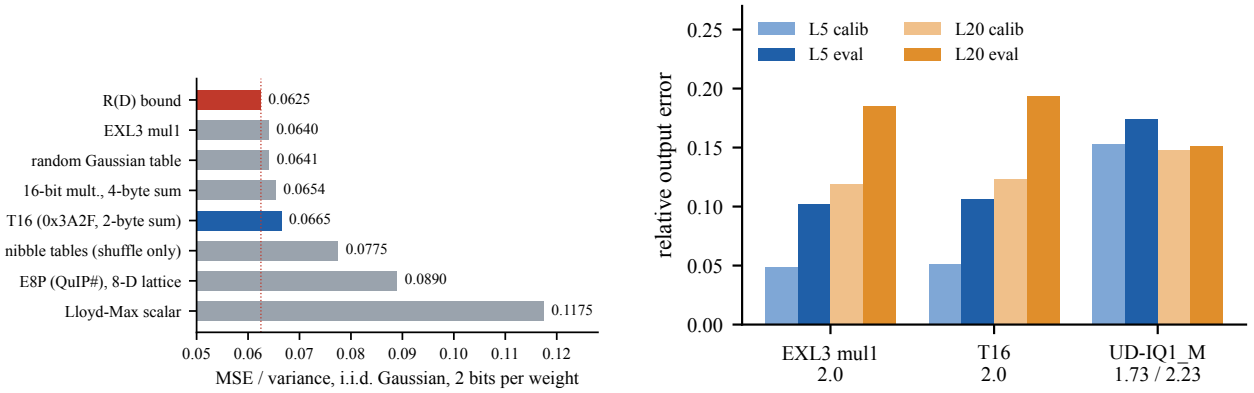


Figure 3: Left: codebook accuracy at 2 bits on i.i.d. Gaussian tiles (Viterbi at the best scale; E8P from QTIP [23]). Right: relative output error of real V4.1 experts (every 8th expert of layers 5 and 20) on calibration tokens and on held-out evaluation tokens, with the full input Hessian. T16 stays within 4% of `mul1`. Both trellis codes lose 1.6–2.4× from calibration to new text, while UD-IQ1_M's experts lose 1.0–1.2×; this is the transfer gap that Sections 5 and 6 address.

Layer, K (gate/up/down)	Tokens	Tensor-core order	Column-major	Ratio	Per expert
5, 1.5/1.5/2	calibration	0.07943	0.07966	1.003	0.98–1.01
5, 1.5/1.5/2	evaluation	0.12765	0.13111	1.027	median 1.005
20, 2/2/2.5	calibration	0.06726	0.06719	0.999	0.99–1.01
20, 2/2/2.5	evaluation	0.09808	0.09678	0.987	median 1.000

Table 2: Tile position order does not affect quality. Relative output error, $\sum\|\hat{y}-y\|^2/\sum\|y\|^2$ over 12 experts per layer, damped Hessian ($\sigma_{\text{reg}} = 2$). Gaussian tiles: 0.07122 vs 0.07124.

At 2 bits T16 takes 29–31% less GPU time than IQ2_XXS and faster than every IQ format at every rate, plus $\sim 6\,\mu$s per call for its two Hadamard transforms and scales, which can be fused into the activation kernels that run anyway. On the CPU T16 beats the UD mix at 1.5 and 2 bits, and at 2.5 bits it is within 3% and bound by RAM.

5 The quantizer

We use ExLlamaV3's quantizer [25] (v1.6.4), patched for T16. Its `mul1` constants become T16's, its tile permutation becomes column-major, and the Hessian stays on the GPU. For each expert matrix it rotates W and the input Hessian H with randomized 128-wide Hadamard transforms, runs LDLQ over 16-row bands with a Viterbi search per 16×16 tile (65,536 states, two passes for tail-biting), and refits the per-channel scales against H. The Hessians come from a calibration chain that runs DeepSeek's reference model one layer at a time over the calibration set and records every expert's routed inputs.

Damping. With the full Hessian, T16 experts fit their calibration tokens closely but lose 1.6–2.4× of their accuracy on new text (Figure 3, right). Each expert sees only the few thousand tokens routed to it, so its Hessian is a noisy estimate. We regularize LDLQ with $\sigma_{\text{reg}} = 2$ (twice the mean diagonal), which keeps the fit general.

A dense fractional-rate Viterbi kernel. Half-integer rates, where positions alternate $\lfloor K \rfloor$ and $\lfloor K \rfloor + 1$ bits, use a generic kernel in ExLlamaV3. That kernel keeps its path costs in global memory, re-decodes all 65,536 codebook values at every step, and stores a 16-bit predecessor per node at the widest stride (16 MB of traceback per tile). ncu showed it memory-bound: DRAM 68% busy, one instruction per cycle, and no eligible warp in 73% of cycles. We rebuilt it in the style of ExLlamaV3's dense integer-rate kernels. Each of 1024 threads owns the same 64 states at every step, whatever the step's width, so the decoded values stay in registers. The costs live in shared memory, one buffer per

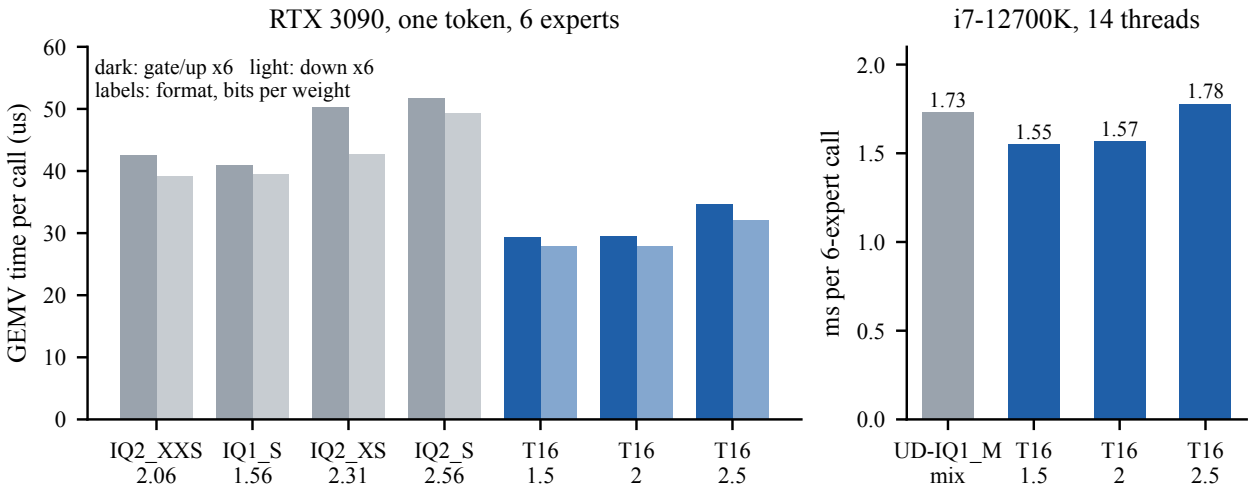


Figure 4: Decode cost per call of 6 experts (one token), measured on the target machine. Left: RTX 3090 GEMV, nsys medians at 285 W over two interleaved rounds; IQ tiles are dsx's kernels for ggml's formats. Right: i7-12700K, 14 threads, interleaved with UD-IQ1_M's layer mix.

node width, and with alternating widths a step reads one buffer and writes the other, so it needs one barrier. The history stores each node's chosen branch, packed by warp ballots into 1 MB per tile. Arithmetic and tie order are unchanged, so the indices are bit-identical: we verified this on Gaussian tiles and on every tensor of three real experts. The new kernel is bound by the shared-memory pipe (79%, 2.5 instructions per cycle). Figure 5 shows the effect: 5.1× at 1.5 bits, 2.1× at 2.5 bits, and per-expert time 16.3 s → 5.0 s at the 1.5-bit layout. That cut the full-model quantization estimate from ∼50 to ∼23 RTX 3090-hours.

5.1 Output shrinkage and a free gain correction

A reconstruction that minimizes squared error is biased toward zero. If Q is the MSE-optimal reconstruction of W from its code, the error is orthogonal to it, $\langle W-Q, Q\rangle = 0$, so the gain is

$$g = \frac{\langle Q, W\rangle}{\langle W, W\rangle} = \frac{\|Q\|^2}{\|W\|^2} = 1 - \frac{\|W-Q\|^2}{\|W\|^2}. \tag{3}$$

The Viterbi search with a refit scale behaves like this. On a layer-5 expert, the relative weight error is 0.151 at 1.5 bits and 0.075 at 2 bits, and the measured gains are 0.862 and 0.929, close to $1-$ the error (0.849 and 0.925). Per matrix, shrinking is the right choice, since it cuts noise more than it adds bias. In a network, however, the bias is the same sign in every expert of every layer, and it compounds through 40 residual blocks.

We correct it in the stored output scales s_v, which costs nothing at run time. Sweeping the gain on a model with T16 experts in layers 0–19 (1.5-bit gate/up) took KL from 0.3145 to 0.2503 at ×1.13 (Figure 6, left). On the full model, the 2-bit gate/up optimum is ×1.04 (KL 0.1962 → 0.1943, right). Down projections did not benefit. The KL-optimal factors sit between the plain gain and the gain weighted by the input Hessian, which is 0.96–0.97 on calibration activations. We read this as one more symptom of the calibration-to-evaluation transfer gap: on held-out activations the shrinkage is larger than the calibration Hessian suggests. The factors were chosen on the evaluation set itself, which we note as a limitation (Section 9). From now on, the worker records both gains for every matrix, so that per-expert corrections can later be fit on held-out activations.

6 Calibration for a 384-expert MoE

Each expert's Hessian is built from the tokens routed to it, so calibration has to cover the experts as well as the domains. The first full run (calib4) used 715K tokens, of which 635K fed the Hessians. It combined coding-agent sessions, agent and code prompts, the model's own reasoning traces, Wikipedia in five languages, the-stack-smol code and UltraChat.

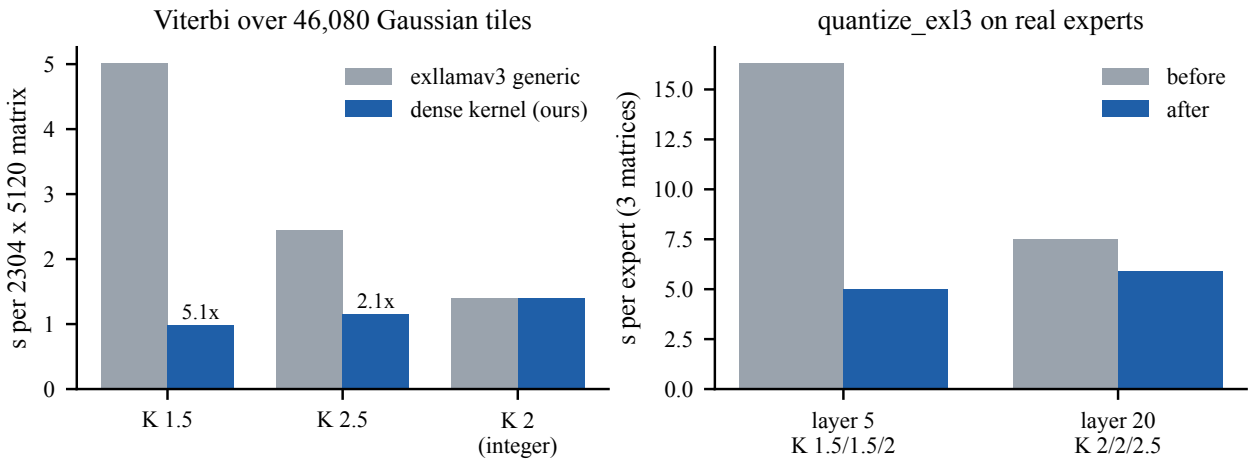


Figure 5: The dense fractional-rate Viterbi kernel (RTX 3090 at 250 W). Left: one 2304×5120 matrix of Gaussian tiles, ExLlamaV3's generic kernel against ours, with identical indices; the integer-rate kernel at $K = 2$ is shown for reference. Right: `quantize_exl3` per expert on real V4.1 experts.

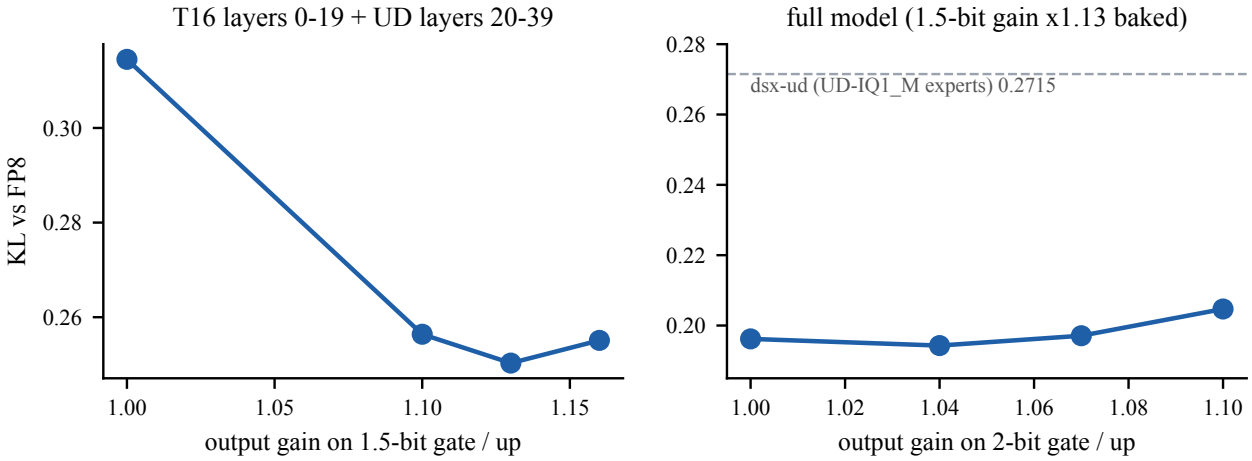


Figure 6: Output-gain sweeps (Lucebox KL vs FP8). Left: T16 experts in layers 0–19 (UD's elsewhere), gain on 1.5-bit gate/up. Right: full T16 model with $\times 1.13$ baked in, gain on 2-bit gate/up; dsx-ud for reference.

calib5. An audit of calib4 found 37K tokens of web text, agent sequences that are mostly prompts and tool output (11K assistant tokens out of 103K), Wikipedia drawn only from alphabetically early articles, and 80K tokens held out for a screening step the trellis pipeline does not use. calib5 drops the web text and uses every sequence for the Hessians. It adds three sources:

- 253K tokens of real coding-agent transcripts, scrubbed of credentials before they left the machine that held them (presigned URLs, environment-style secrets, keys and tokens, emails; checked with an independent scan) and rendered with DeepSeek's V4.1 chat encoder, with tool calls and thinking blocks;
- SWE-smith agent trajectories [21], converted to the model's tool-call format;
- R1-0528 reasoning traces [1].

It also samples Wikipedia at random from ten languages and keeps the model's own reasoning traces. Every text is filtered against the evaluation set with 50-character shingles. Every rendered window fits one 2048-token row.

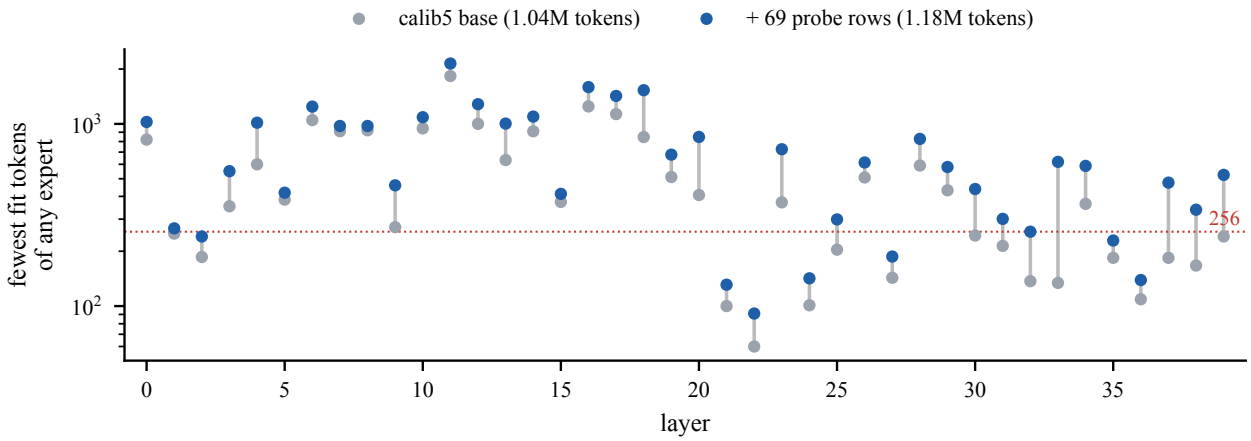


Figure 7: Fewest calibration tokens of any expert, per layer, before and after the probe top-up (calib5). The probes were mostly agent trajectories (28 rows), code (15), reasoning (13) and Chinese and Japanese Wikipedia (6).

Coverage top-up. A routing-only pass over the base set plus a 428K-token candidate pool counts each (layer, expert)'s tokens. A greedy selector then adds the pool rows that fill the most remaining deficit below 256 tokens, under a cap that keeps the set at or below 1.25M tokens. The rarest expert in calib4 (layer 22, expert 310, 12 tokens) fired on the concept of dividing (the character *fēn* and "divided into"), almost only in Chinese and Japanese Wikipedia, so the pool was weighted toward those languages and code. The selector added 69 rows, giving 1,177,600 tokens. It reduced the experts below 256 tokens from 24 to 8 and raised the worst per-layer minimum from 60 to 91 (Figure 7). It stopped before its cap because the pool held nothing for the last 8 experts; those need targeted text. On the Lucebox evaluation the calib5 model scores KL 0.202 against calib4's 0.196, with slightly higher top-1 agreement (81.3% vs 80.9%). Two caveats apply. The output gains were tuned on the calib4 model and reused unchanged. And calib5 removed the web text that calib4 contained, which is the closest domain to this evaluation's text. calib5 was built for agentic use, which this evaluation does not measure, so we report it as an open result.

7 Results

Setup. Experts use 1.5/1.5/2 bits (gate/up/down) in layers 0–19 and 2/2/2.5 in layers 20–39, about 1.9 bits per weight, sized to match UD-IQ1_M. The 15,360 experts take 131 GB. Dense weights are Q8_0, and the Engram table is IQ4_NL (or Q6_K). The whole model ships as one GGUF file with the tokenizer inside [20]. Quality is measured on the Lucebox evaluation set [13]: 8×1024 tokens scored against the FP8 model's top-256 log-probabilities (KL divergence, top-1 agreement, perplexity). Speed is measured on 2×RTX 3090 (250/280 W), an i7-12700K and 128 GB of DDR4.

Quality. Table 3 and Figure 8 summarize quality. At the same size as dsx-ud, T16's experts lower KL by 28% (0.272 → 0.194) and raise top-1 agreement from 79.0% to 81.4%. That is about the quality of the 178 GB Maya-S build in 141 GB. Through the decode path, the gap is the same (0.303 → 0.205). Relative to dsx's earlier scalar formats at 148 GB, KL drops by 48%. The gain correction is essential: without it, T16 experts in layers 0–19 made the model worse than UD's experts there (KL 0.3145 against dsx-ud's 0.2715), and with it they made it better (0.2503).

Speed. On the same machine, with dsx-ud measured in the same session (Table 4), T16 decodes the same 2,046 teacher-forced tokens 15% faster (215 s vs 253 s). Free-running decode after a 3K-token prompt reaches 34.9 tokens/s against 27.9–28.0. Per layer, T16's experts compute 20–34% faster on the CPU (1.21/1.46 ms per 6-expert call vs 1.83, 14 threads) and 22% faster on the GPU (125 vs 161 μs). Prefill is 9% slower for now. An nsys profile of a 3K-token prefill shows the first T16 dequantize-to-fp16 kernel taking 3.6 s of GPU0's 6.6 s of kernel time (127 μs per call). A rewritten kernel is 2.5–3.0× faster on an RTX 3090 and bit-identical. In the same session it raises 3K-token prefill from 388 to 415–428 tokens/s, closing most of the gap to dsx-ud.

Build	Experts	Runtime	GB	KL ↓	Top-1 ↑	PPL ↓
FP8 (reference)	FP4 / FP8			0	100%	8.56
dsx-2bit v4b	scalar ternary / 2 / 3-bit	dsx	148	0.373	74.4%	10.68
Unsloth UD-IQ1_M	IQ1_S/IQ2_* mix	llama.cpp	∼140	0.275	78.8%	10.11
dsx-ud	UD-IQ1_M's experts	dsx	142	0.272	79.0%	10.17
Maya-S	llama.cpp mix	llama.cpp	178	0.195	82.0%	9.49
dsx-t16 v1 (IQ4_NL Engram)	**T16**	dsx	**141**	**0.196**	**80.9%**	**9.54**
dsx-t16 v1 (Q6_K Engram)	**T16**	dsx	**141**	**0.194**	**81.4%**	**9.60**
dsx-t16 v2, calib5 (IQ4_NL Engram)	T16	dsx	141	0.202	81.3%	9.58
Decode path (one token at a time, 2,046 tokens)						
dsx-ud		dsx	142	0.303	80.4%	9.03
dsx-t16 v1		dsx	**141**	**0.205**	**82.0%**	**8.65**

Table 3: Quality against the FP8 model (Lucebox 1017 evaluation). GB counts experts and dense weights, without the Engram table. T16 v1 uses calib4, v2 calib5 with v1's output gains; v1 and v2 were measured in one session.

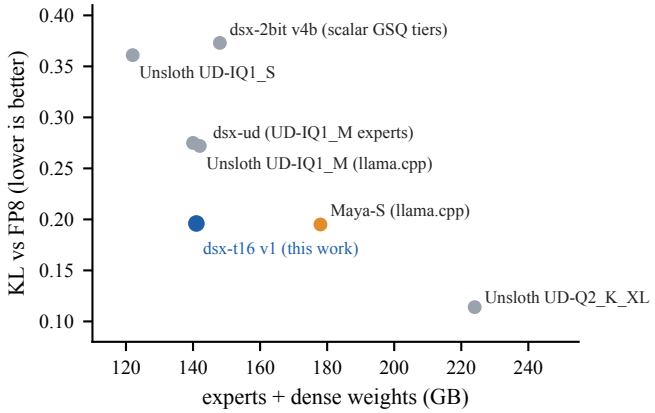


Figure 8: Quality per byte on the Lucebox evaluation (KL vs FP8, experts and dense weights without the Engram table).

Quantization cost. The first full run quantized all 15,360 experts on one rented RTX PRO 6000 (96 GB) in 6.1 hours. The calibration chain ran alongside two quantization workers under NVIDIA MPS (2.45–2.6 s per expert each), and layers were uploaded as they finished, at $2.61/h for $16.00, plus $0.65 for rentals that failed a network check. A third worker added no throughput. The calib5 requant split the layers into two blocks on two pods, each running the chain from layer 0 to its block's end. It took 6.3 hours of wall time and $33.20; the larger calibration set raised the time per expert to 2.9–3.0 s.

8 Packaging

T16 ships inside GGUF as dsx-specific tensor types (ids 1010–1012, one per rate), so the model, tokenizer and metadata stay in one file set. Hugging Face's GGUF parser accepts the unknown ids and counts parameters from the shapes; llama.cpp's tools reject them. We decided against an EXL3-layout export. ExLlamaV3 identifies its codebook by the presence of a `mul1` tensor and assumes tensor-core position order, so it would load T16 files and silently produce wrong weights. A byte-exact specification (Section A) and a NumPy reference decoder, which reproduces ExLlamaV3's reconstruction bit for bit on golden files, document the format.

2×RTX 3090 + i7-12700K + 128 GB	dsx-t16	dsx-ud
Decode, the same 2,046 tokens (teacher-forced)	**215 s**	253 s
Decode, 3K-token prompt + 300 tokens	**34.9 tok/s**	27.9–28.0 tok/s
Decode with expert deferral (`-defer-rank 3`)	**36.8 tok/s**	36.1–36.2 tok/s
Prefill, 3K-token prompt	361–367 tok/s	393–409 tok/s
Expert compute per layer, CPU (6 experts, 14 threads)	**1.21 / 1.46 ms**	1.83 ms
Expert compute per layer, GPU (6 resident experts)	**125 μs**	161 μs

Table 4: Speed. Free-running decode generates different text per build, which moves the share of experts the CPU computes; the teacher-forced row is the like-for-like comparison.

9 Limitations

- The output gains ($\times 1.13$, $\times 1.04$) were chosen on the evaluation set used for the results, so the reported KL is slightly optimistic. Fitting per-expert gains on held-out activations is future work.
- All quality numbers come from one evaluation set of 8×1024 tokens. We did not run downstream task suites.
- Trellis experts still transfer from calibration to new text worse than UD's codebook experts per unit of calibration error. Damping and broader calibration narrow the gap; YAQA-style [24] Hessians or fine-tuning might close it, but at a cost beyond our budget.
- T16 is specific to dsx. It is not loadable by ExLlamaV3 or llama.cpp, and quantization needs a GPU because the Viterbi search runs over 65,536 states.
- The broader calibration set (calib5) did not lower KL on this evaluation (0.202 vs 0.196) with reused gains. Whether it helps agentic workloads, as intended, needs an evaluation on such text.

10 Conclusion

For a 552B MoE on a desktop, the expert format is a systems problem: it has to decode at memory speed on two kinds of processor. Designing the codebook from a measured instruction budget gives T16, which costs 4% more MSE than ExLlamaV3's codebook on Gaussian data but decodes faster than every IQ format on both the GPU and the CPU. Combined with ExLlamaV3's quantizer, a damped Hessian, a free output-gain correction and routing-aware calibration, it lowers KL by 28% at equal size and speeds up decode, on hardware that costs less than a single datacenter GPU.

Acknowledgments. We thank DeepSeek for releasing the model, turboderp for ExLlamaV3, the QTIP authors for bitshift trellises, the llama.cpp and Unsloth teams for their formats and builds, Lucebox for the evaluation set, and the Xolotl/Strata contributors for the V4.1 GGUF tooling. This work was carried out with several Claude Code (Anthropic) agent sessions working under the author's direction.

References

[1] a-m-team. AM-DeepSeek-R1-0528-Distilled. https://huggingface.co/datasets/a-m-team/AM-DeepSeek-R1-0528-Distilled, 2025.

[2] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Pashmina Cameron, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. QuaRot: Outlier-free 4-bit inference in rotated LLMs. In *Advances in Neural Information Processing Systems*, 2024. arXiv:2404.00456.

[3] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher De Sa. QuIP: 2-bit quantization of large language models with guarantees. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2307.13304.

[4] Hongtao Chen et al. KTransformers: Unleashing the full potential of CPU/GPU hybrid inference for MoE models. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2025.

[5] Alireza Dadgarnia, Soroush Tabesh, Mahdi Nikdan, Michael Helcig, Eldar Kurtic, Maximilian Kleinegger, and Dan Alistarh. GSQ: Highly-accurate low-precision scalar quantization for LLMs via Gumbel-softmax sampling. *arXiv preprint arXiv:2604.18556*, 2026.

[6] DeepSeek-AI. DeepSeek-V4.1-Flash-1017. https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash-1017, 2026.

[7] Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, Elias Frantar, Artem Babenko, and Dan Alistarh. Extreme compression of large language models via additive quantization. In *International Conference on Machine Learning*, 2024. arXiv:2401.06118.

[8] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations*, 2023. arXiv:2210.17323.

[9] Georgi Gerganov et al. llama.cpp and ggml. https://github.com/ggml-org/llama.cpp, 2026.

[10] Iwan Kawrakow. ik_llama.cpp: IQ*_KT trellis quantization types. https://github.com/ikawrakow/ik_llama.cpp, PRs 113, 505, 511, 2025.

[11] Deokjae Lee and Hyun Oh Song. Q-Palette: Fractional-bit quantizers toward optimal bit allocation for efficient LLM deployment. In *Advances in Neural Information Processing Systems*, 2025. arXiv:2509.20214.

[12] Yifei Liu, Jicheng Wen, Yang Wang, Shengyu Ye, Li Lyna Zhang, Ting Cao, Cheng Li, and Mao Yang. VPTQ: Extreme low-bit vector post-training quantization for large language models. In *Proceedings of EMNLP*, 2024. arXiv:2409.17066.

[13] Lucebox. DeepSeek-V4.1-Flash-1017-ROCMFP-GGUF and its evaluation set. https://huggingface.co/Lucebox/DeepSeek-V4.1-Flash-1017-ROCMFP-GGUF, 2026.

[14] Vladimir Malinovskii, Denis Mazur, Ivan Ilin, Denis Kuznedelev, Konstantin Burlachenko, Kai Yi, Dan Alistarh, and Peter Richtárik. PV-Tuning: Beyond straight-through estimation for extreme LLM compression. In *Advances in Neural Information Processing Systems*, 2024. arXiv:2405.14852.

[15] Vladimir Malinovskii, Andrei Panferov, Ivan Ilin, Han Guo, Peter Richtárik, and Dan Alistarh. Pushing the limits of large language model quantization via the linearity theorem. *arXiv preprint arXiv:2411.17525*, 2024.

[16] Michael W. Marcellin and Thomas R. Fischer. Trellis coded quantization of memoryless and Gauss-Markov sources. *IEEE Transactions on Communications*, 38(1):82–93, 1990.

[17] Niko1221 and the Strata contributors. Xolotl / strata: DeepSeek-V4.1 GGUF tools. https://github.com/Indras-Mirror/Xolotl, 2026.

[18] Xiaofan Que, Nir Elkayam, Spandan Pyakurel, Shuokai Pan, and Dibakar Gope. XOR-Trellis: Ultra-low-complexity dequantization and curvature-aware Hadamard-free LLM quantization. *arXiv preprint arXiv:2610.00432*, 2026.

[19] Aleksey Smolenchuk. dsx: DeepSeek-V4.1-Flash-1017 on a desktop. https://github.com/lxe/dsx, 2026.

[20] Aleksey Smolenchuk. DeepSeek-V4.1-Flash-1017-dsx-t16. https://huggingface.co/lxe/DeepSeek-V4.1-Flash-1017-dsx-t16, 2026.

[21] SWE-bench team. SWE-smith trajectories. https://huggingface.co/datasets/SWE-bench/SWE-smith-trajectories, 2025.

[22] Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa. QuIP#: Even better LLM quantization with Hadamard incoherence and lattice codebooks. In *International Conference on Machine Learning*, 2024. arXiv:2402.04396.

[23] Albert Tseng, Qingyao Sun, David Hou, and Christopher De Sa. QTIP: Quantization with trellises and incoherence processing. In *Advances in Neural Information Processing Systems*, 2024. arXiv:2406.11235.

[24] Albert Tseng, Zhaofeng Sun, and Christopher De Sa. Model-preserving adaptive rounding. In *International Conference on Machine Learning*, 2026. arXiv:2505.22988.

[25] turboderp. ExLlamaV3. https://github.com/turboderp-org/exllamav3, v1.6.4, commit da68d57, 2026.

[26] Unsloth AI. Unsloth dynamic 2.0 GGUFs and DeepSeek-V4.1-Flash-1017-GGUF. https://huggingface.co/unsloth/DeepSeek-V4.1-Flash-1017-GGUF, 2026.

[27] Andrew J. Viterbi. Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *IEEE Transactions on Information Theory*, 13(2):260–269, 1967.

A T16 format specification

An expert matrix with k inputs and n outputs (k, n multiples of 128) is stored as:

- **Tiles.** Tile (k_b, n_b) covers inputs $16k_b \dots 16k_b{+}15$ and outputs $16n_b \dots 16n_b{+}15$. Tiles are stored by tile column (index $n_b \cdot k/16 + k_b$), $32K$ bytes each (48/64/80 bytes at 1.5/2/2.5 bits).
- **Bit stream.** Each tile is one circular stream of $256K$ bits; byte 0 holds bits 0–7, most significant bit first.
- **Positions.** Position t is input t mod 16 and output $\lfloor t/16 \rfloor$. At integer K, position t's bits end at $(t+1)K$. At half-integer K, even positions take $\lfloor K \rfloor$ bits and odd positions one more, ending at $(t+1)\lfloor K \rfloor + \lfloor (t+1)/2 \rfloor$.
- **State and value.** The state is the 16 bits ending at the position's end bit, MSB first, wrapping around the stream. $p = (\text{state} \cdot \texttt{0x3A2F}) \bmod 2^{16}$, $b = (p \,\&\, \texttt{0xFF}) + (p \gg 8)$, and the value is fp16(fma(fp16($1024 + b$), fp16 `0x20e6`, fp16 `0xca1e`)): the exact product and sum, rounded once.
- **Scales and transforms.** s_u (fp16, $[k]$) and s_v (fp16, $[n]$); $y = s_v \odot \mathscr{H}_{128}(\mathscr{H}_{128}(x \odot s_u)\, W_{\text{inner}})$ with orthonormal Walsh-Hadamard blocks of 128. Output gains, where used, are already multiplied into s_v.

ExLlamaV3's own layout is `int16 [k/16][n/16][16K]`, read as little-endian 32-bit words, MSB first. The dsx layout reorders the tiles by column and byte-swaps each 32-bit word. The reference code is `core/src/t16.rs`, `quant/t16/t16_ref.py` and `docs/t16-format.md` in the dsx repository [19].